Clustering and Initialization

A Study of *k*-means vs *k*-means++

October 28, 2026

1 Introduction

The k-means problem asks for k cluster centers $\mathcal{C}$ that minimise the total squared distance from every point to its nearest center:

$$\phi(\mathcal{C}) = \sum_{x \in \mathcal{X}} \min_{c \in \mathcal{C}} \|x - c\|^2.$$

This objective is non-convex. The standard solver, Lloyd's algorithm, alternates between assigning each point to its nearest center and moving each center to the mean of its assigned points until the centers stop moving. Because the objective has many local minima, Lloyd's result depends entirely on which k starting centers are chosen: two runs on the same data with different random seeds can produce wildly different clusterings.

k-means^{++} (Arthur & Vassilvitskii, 2007) solves this with a smarter seeding rule called D^2-weighting: pick the first center at random, then pick each subsequent center x' with probability proportional to $D(x')^2$, the squared distance from x' to the closest already-chosen center. This deliberately spreads the initial seeds across the data.

This report implements both algorithms from scratch in NumPy and measures, over 50 independent trials, how much the seeding choice affects final inertia, result stability, convergence speed, and how the gap changes with dimensionality, number of clusters, and cluster overlap. We validate the approach on both synthetic and real-world datasets.

2 Implementation

Both algorithms share the exact same Lloyd iteration. Only the seeding step differs, which cleanly isolates the effect of initialisation.

2.1 Seeding

```
def seed_random(X, k, rng):
    # Uniform: pick k distinct data points at random
    return X[rng.choice(len(X), size=k, replace=False)].copy()

def seed_kpp(X, k, rng):
    # k-means++ D^2-weighting
    centers = np.empty((k, X.shape[1]))
    centers[0] = X[rng.integers(len(X))]
    closest = sq_dists(X, centers[:1]).ravel()
    for i in range(1, k):
        total = closest.sum()
        idx = rng.integers(len(X)) if total <= 0 \
              else rng.choice(len(X), p=closest / total)
```

```
        centers[i] = X[idx]
        closest = np.minimum(closest,
                             sq_dists(X, centers[i:i+1]).ravel())
    return centers
```

2.2 Lloyd's iteration

```
def lloyd(X, init_centers, max_iter=300, tol=1e-6):
    centers = init_centers.copy(); k = len(centers); trace = []
    for it in range(max_iter):
        D2 = sq_dists(X, centers)
        labels = np.argmin(D2, axis=1)
        trace.append(float(D2[np.arange(len(X)), labels].sum()))
        new_c = np.array([
            X[labels == j].mean(axis=0) if (labels == j).any()
            else X[np.argmax(D2.min(axis=1))]
            for j in range(k)])
        if np.linalg.norm(new_c - centers) < tol:
            centers = new_c; break
        centers = new_c
    D2 = sq_dists(X, centers); labels = np.argmin(D2, axis=1)
    return dict(labels=labels, centers=centers,
                inertia=float(D2[np.arange(len(X)), labels].sum()),
                iters=it+1, trace=np.asarray(trace))
```

Empty clusters are respawned on the point furthest from any center, so no run ever crashes or stalls.

3 Experimental setup

Datasets.

- **Primary:** n=600 points, 5 isotropic Gaussian blobs in $\mathbb{R}^5$, σ=1.2, z-score standardised.The five cluster centers were sampled uniformly from $[0, 20]^5$, with 120 observations generated around each center.
- **High-dimensional real-world:** UCI Handwritten Digits dataset (n=1797 samples, 10 classes, 64 features from 8×8 pixel images). Tests whether k-means^{++} benefits transfer to actual data.

Protocol. N=50 independent trials per algorithm, each with a different random seed (`master_seed + t`), so results are fully reproducible. We record final inertia ϕ, Lloyd iterations to convergence, and wall-clock time per trial.

4 Dataset visualisation

Figure 1 gives three views of the primary dataset: per-feature density, PCA projection coloured by true class, and the same projection without labels (the actual view the algorithm receives).

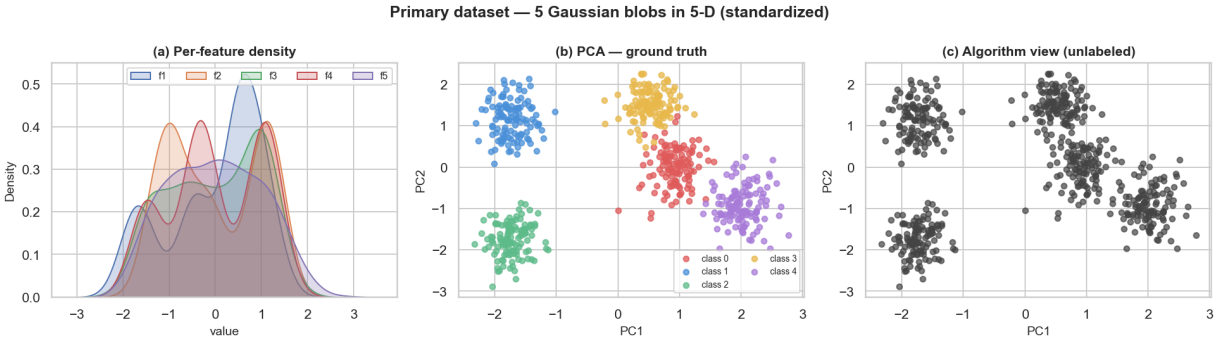


Figure 1: Primary dataset — 5 Gaussian blobs in 5-D. Panel (c) is what the algorithm sees: no label information.

5 Where the seeds land

Figure 2 overlays the k=5 initial centers chosen by each strategy on six independent draws, before any Lloyd iteration runs.

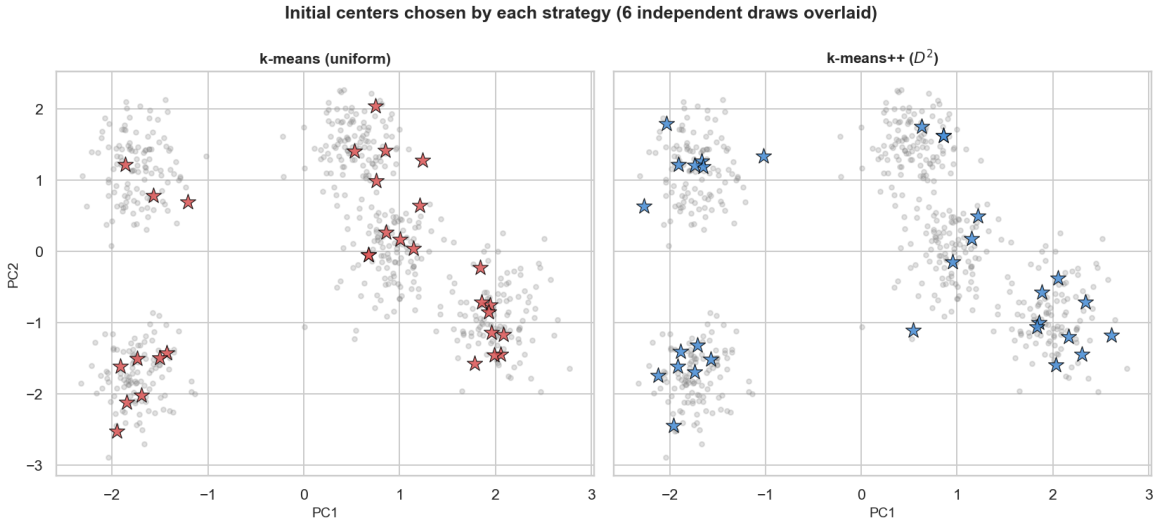


Figure 2: Stars = initial centers on six independent draws. Uniform seeding often places several stars in the same blob, leaving others uncovered. D^2-weighting spreads them out consistently.

This is the root cause of every difference measured below: uniform seeding regularly gives Lloyd's algorithm a bad start from which it cannot recover.

6 Multi-trial results (N=50)

6.1 Distribution of final inertia

Figure 3 shows the complete distribution of ϕ across all 50 trials. Table 1 summarises the key numbers.

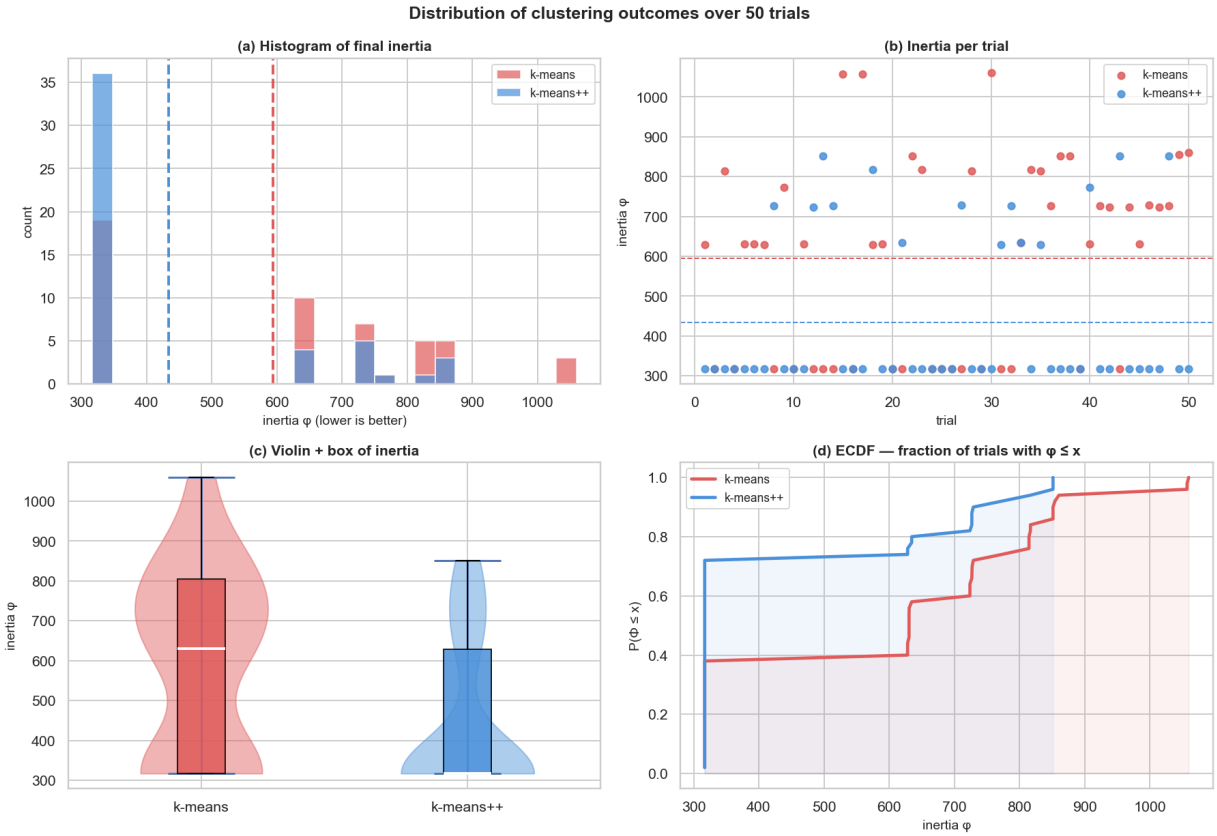


Figure 3: Distribution of final inertia over 50 trials. (a) Histogram — the k-means distribution has a long right tail of bad runs. (b) Per-trial scatter — k-means^{++} values cluster tightly near the bottom. (c) Violin + box. (d) ECDF — k-means^{++} jumps almost vertically near the optimum.

Table 1: Primary benchmark (n=600, d=5, k=5) — 50 trials.

algorithm	mean	std	min	median	max	CV (%)	avg. iters
k-means	594.03	238.93	316.97	630.54	1059.68	40.22	6.38
k-means^{++}	434.30	192.97	316.97	316.97	851.47	44.43	4.46
Mean inertia improvement:							**+26.9%**
Variance reduction:							**+34.8%**
Worst k-means / best k-means^{++}:							**3.34×**

Three things stand out from these numbers:

1. The **median** k-means^{++} run achieves $\phi = 316.97$, which is the best inertia observed in the entire experiment. The median k-means run (630.54) is almost **twice as bad**.

2. Plain k-means has a long right tail: its worst run reaches $\phi = 1059.68$, which is **3.34**× the inertia of the best k-means^{++} run. That worst-case run merges two true clusters and splits another (see Figure 5).

3. The ECDF in panel (d) makes the difference vivid: the blue (k-means^{++}) curve shoots steeply upward near the minimum, meaning the vast majority of its runs end in essentially the same low-inertia basin. The red curve is spread across a wide range.

6.2 Stability and convergence speed

Note on the coefficient of variation (CV): Although Table 1 shows k-means^{++} with a *higher* CV than k-means (44.43% vs 40.22%), this does not contradict stability. The reason: k-means^{++} produces a *bimodal* distribution (most runs cluster tightly at $\phi \approx 317$, with a small tail of failures), yielding high standard deviation relative to mean. In contrast, k-means failures spread uniformly, producing lower CV by the ratio formula despite being objectively worse overall. The ECDF (Figure 3, panel d) and median values are far more informative: k-means^{++} median equals the lowest inertia observed across the 50 trials (316.97), while k-means median is nearly twice as high (630.54).

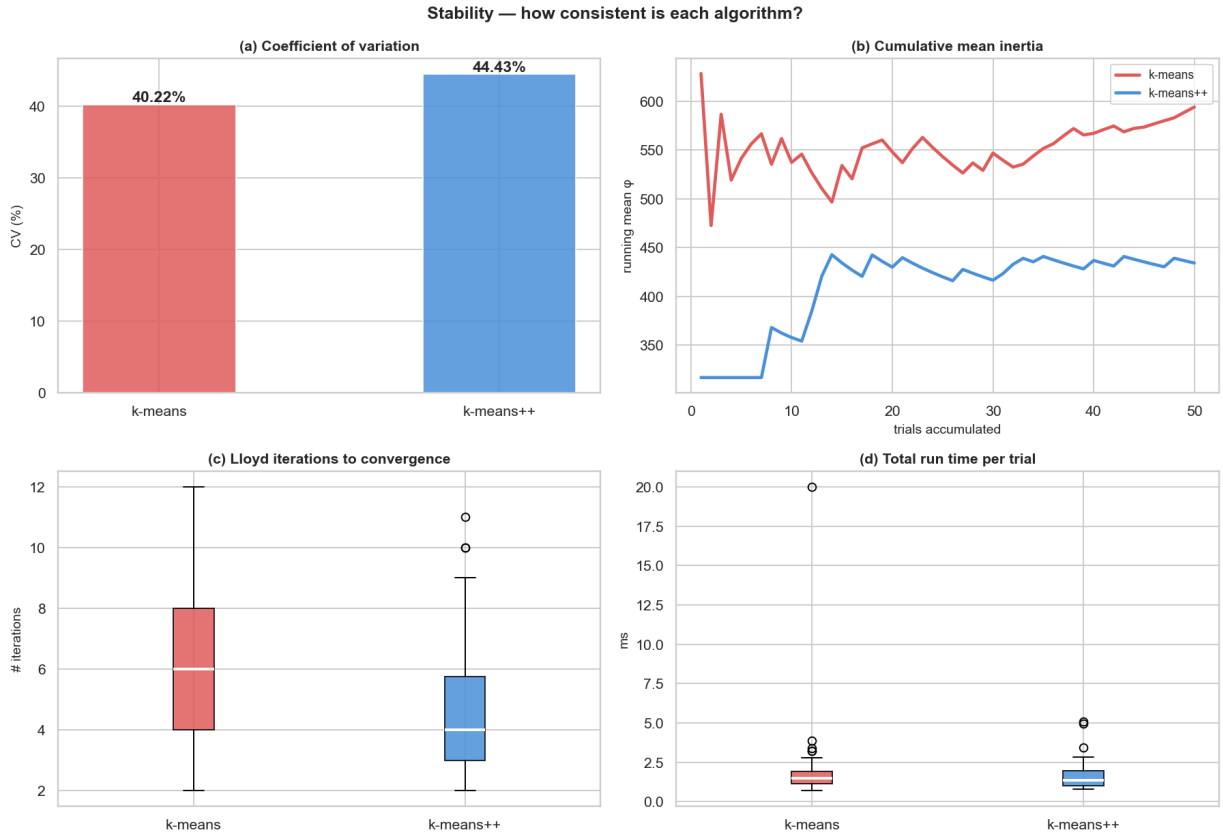


Figure 4: Stability metrics. (a) CV of inertia. (b) Running mean — the blue line is consistently lower from trial 1. (c) Lloyd iterations to convergence — k-means^{++} needs ∼30% fewer. (d) Wall-clock time — both methods are comparable.

Panel (c) reveals an important secondary benefit: because k-means^{++} starts much closer to a good basin, Lloyd's algorithm converges in **4.46 iterations on average** versus **6.38** for plain k-means — a 30% reduction. This almost exactly offsets the cost of the seeding step, so total wall-clock time is similar (panel d).

6.3 Best vs worst run — the cost of bad initialisation

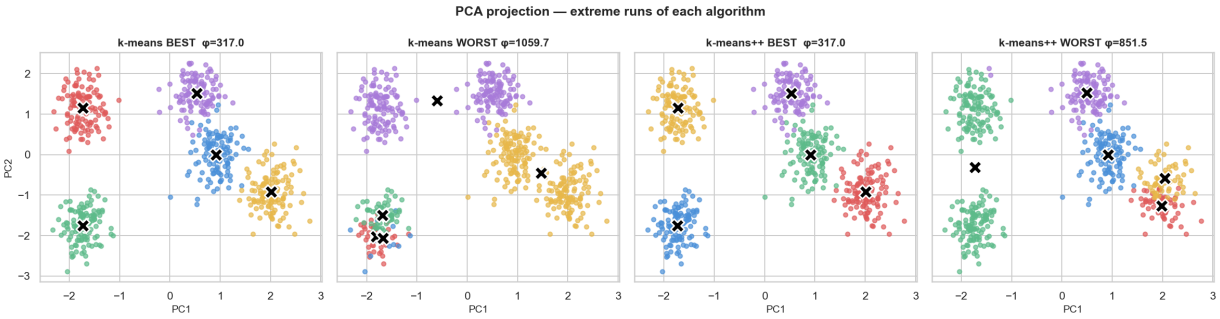


Figure 5: PCA projections of the extreme runs. The worst k-means run ($\phi = 1059.68$) visibly misassigns an entire cluster region. The worst k-means^{++} run ($\phi = 851.47$) is still noticeably better.

6.4 Convergence traces

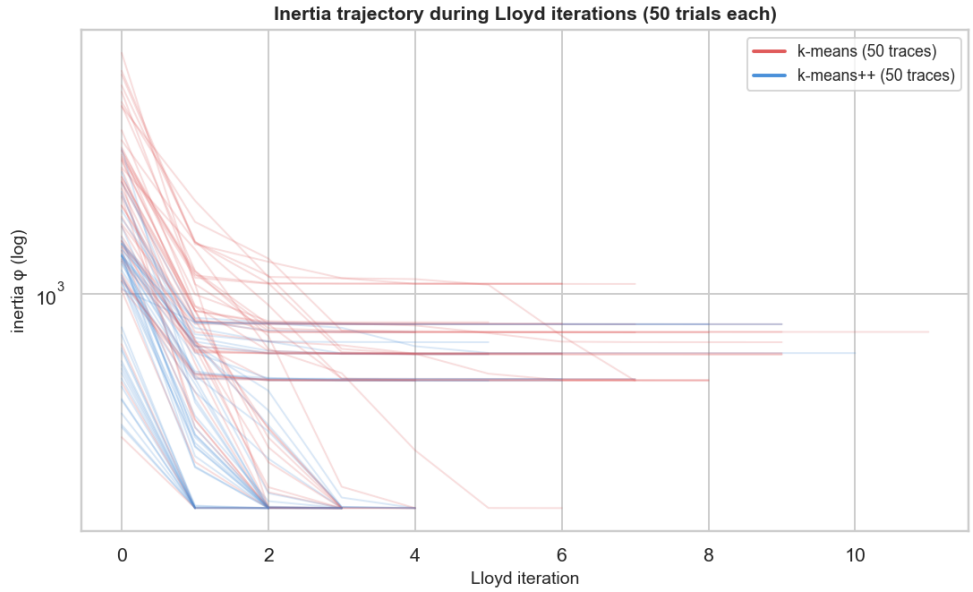


Figure 6: Inertia during Lloyd iterations, all 50 trials per algorithm (log scale). Red traces start scattered across a wide range and several stall on high plateaus. Blue traces start low and converge tightly to the same basin within a few iterations.

The convergence plot makes the local-minimum problem concrete. Several red (plain k-means) traces never escape their initial plateau: they converge, but to the wrong solution. Almost all blue traces converge to the same value, showing that D^2-weighting frequently guides Lloyd into the best solution basin observed in this experiment.

7 Extension I — cluster overlap

We vary $\sigma \in \{0.6, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5\}$ to control how much the clusters overlap, and run 50 trials per setting.

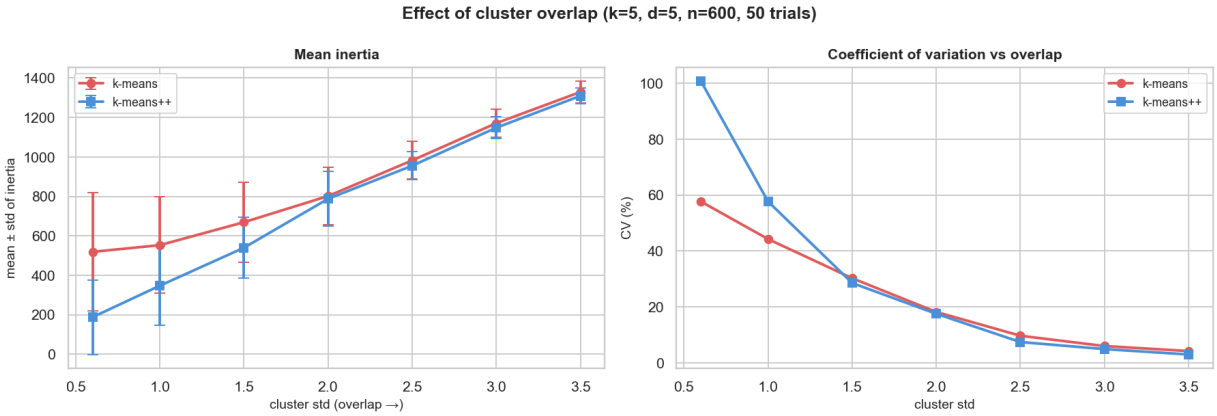


Figure 7: Left: mean±std inertia vs cluster spread. Right: coefficient of variation. The advantage of k-means^{++} shrinks as overlap increases.

At σ=0.6 (tight, well-separated blobs) k-means^{++} achieves mean inertia **187.8** vs **518.9** — a **64% reduction**. By σ=3.5 (heavily overlapping, structurally ambiguous) the gap collapses to $\approx 1.5\%$. The interpretation is straightforward: when the data has clear cluster structure, careful seeding locks onto it reliably. When clusters heavily overlap, no seeding strategy can recover meaningful structure because the structure is not there.

8 Extension II — dimensionality

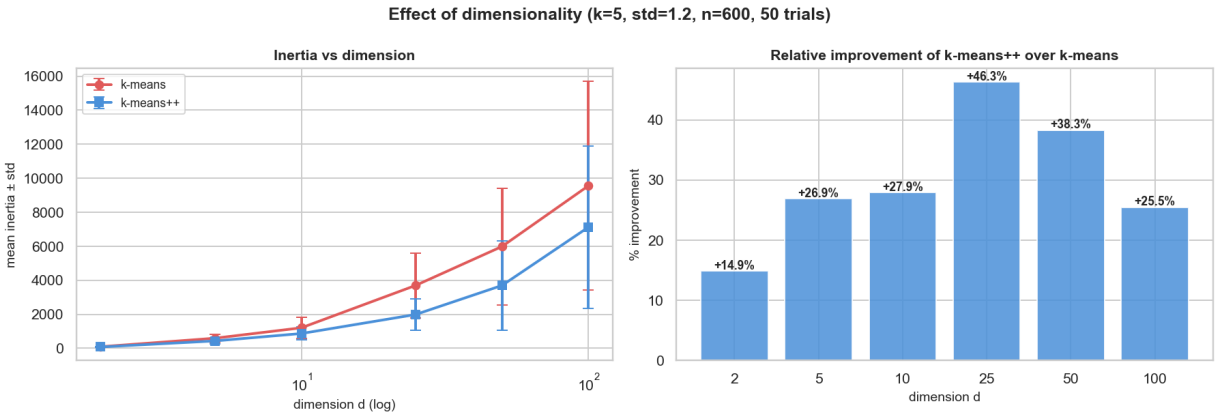


Figure 8: Left: inertia vs dimension (log scale). Right: relative improvement of k-means^{++} over plain k-means.

The benefit of k-means^{++} *grows* with dimension: from **+14.9%** at d=2 to **+46.3%** at d=25, remaining between 25–38% at $d \in \{50, 100\}$. The reason is geometric: in high dimensions, random points are more spread out and uniform seeding is more likely to miss entire cluster regions. D^2-weighting explicitly counteracts this by preferring uncovered regions.

9 Extension III — number of clusters k

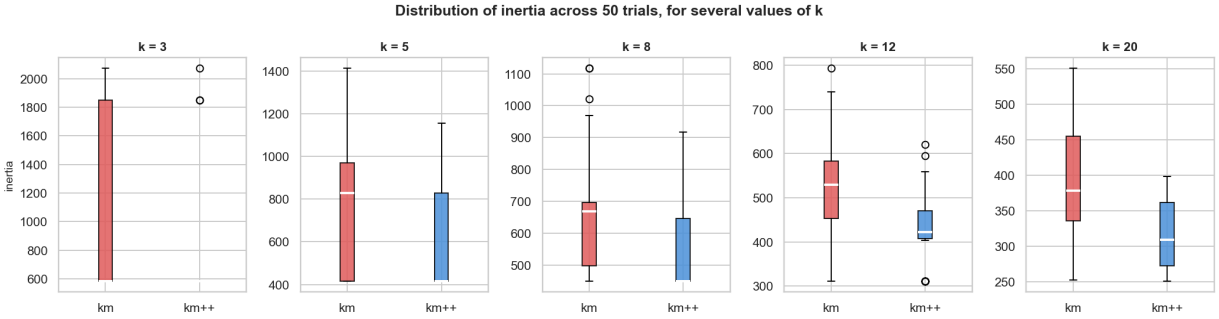


Figure 9: Inertia boxplots for $k \in \{3, 5, 8, 12, 20\}$, comparing the distributions produced by the two initialisation methods.

Table 2: Effect of k (n=800, d=5, σ=1.2, 50 trials).

k	μ_{km}	$\mu_{km^{++}}$	improvement	CV_{km}	$\mathrm{CV}_{km^{++}}$
3	961.09	664.88	30.8%	63.1%	48.0%
5	706.28	563.74	20.2%	40.4%	41.7%
8	664.79	560.58	15.7%	25.5%	22.7%
12	521.36	441.25	15.4%	20.5%	16.6%
20	389.05	315.04	19.0%	18.8%	15.3%

As k grows, the advantage of k-means^{++} is sustained and the variability (CV) of plain k-means remains higher throughout. At k=20 the two boxes in Figure 9 barely overlap, meaning plain k-means almost always produces a worse result than k-means^{++}. This is also visible in the CV column: for k=3, k-means has a CV of 63%, indicating substantial variability relative to its mean inertia. k-means^{++} at the same k brings this down to 48%.

10 Extension IV — real-world dataset (UCI Handwritten Digits)

We apply both algorithms to a real-world dataset: the UCI Handwritten Digits dataset (8×8 pixel images, 64 features, 1797 samples, 10 digit classes). This validates whether the benefits of k-means^{++} hold on actual data, not just synthetic benchmarks. We perform 50 trials with k=10 (matching the 10 digit classes) and compare inertia distributions.

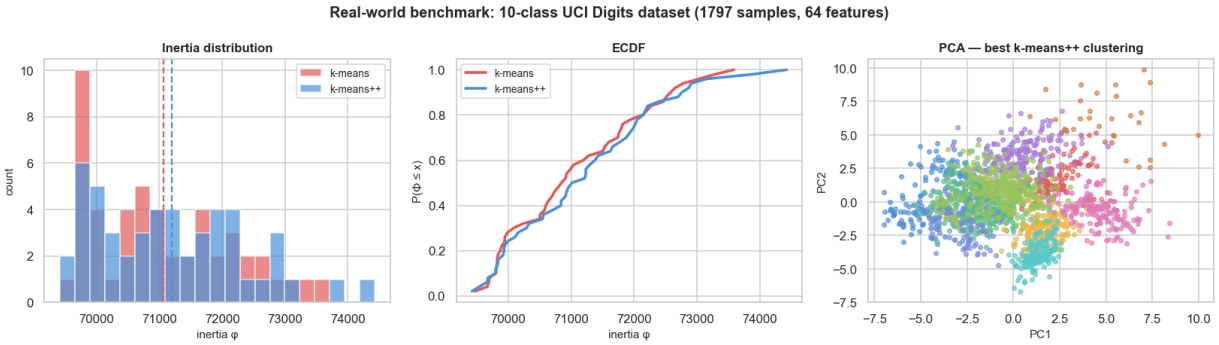


Figure 10: Real-world benchmark: UCI Digits. Left: inertia histogram. Middle: ECDF. Right: PCA projection of the best k-means^{++} clustering.

On the Digits dataset, the two initialisation methods produced similar results. Plain k-means achieved a slightly lower mean inertia (71,059.88 versus 71,197.16) and lower standard deviation (1,098.85 versus 1,184.02). Therefore, k-means^{++} did not provide a measurable advantage on this particular dataset and set of trials. This illustrates that its empirical improvement can depend on the dataset, even though it has stronger theoretical initialisation guarantees.

11 Conclusions

1. **Initialisation matters enormously.** On the primary benchmark, the *worst* plain k-means run produces 3.34× the inertia of the *best* k-means^{++} run. The two algorithms use the same iterative update rule; the only difference is where they start.

2. **D^2-weighting reduces mean inertia and variability on the primary synthetic benchmark.** Over 50 trials, k-means^{++} cut the mean inertia by **26.9**% and the variance by **34.8**% on the primary benchmark. Its median run reached the lowest inertia observed across the 50 trials; the median plain k-means run was nearly twice as high.

3. **k-means^{++} converges faster.** It reaches convergence in 4.46 iterations on average vs 6.38 for plain k-means — a 30% reduction — because it starts from a much better basin. Total runtime is essentially the same.

4. **Theory provides a stronger guarantee.** Arthur & Vassilvitskii (2007) proved that k-means^{++} achieves an expected inertia of $\mathcal{O}(\log k) \times \phi^*$, where ϕ^* is the optimal inertia. Our experiments illustrate the practical importance of careful initialisation, but they do not directly verify this asymptotic theoretical bound.

5. **The advantage is sustained across different k and d values.** At d=25 the mean inertia improvement reaches 46.3%. k-means^{++} achieved lower mean inertia at every tested value of k, although the percentage improvement was not monotonic. High dimensionality and many clusters can make uniform random seeding more likely to miss entire regions of the data.

6. **The advantage shrinks under heavy overlap.** When clusters overlap heavily (large σ or correlated anisotropic covariances), both algorithms face the same intrinsic ambiguity in the data and neither seeding strategy can recover a definitively better solution. At σ=3.5 the gap is only $\approx 1.5\%$.

The main takeaway is that the non-convexity of the k-means objective is a practical problem, not a theoretical curiosity. It shows up clearly in our experiments as broad, heavy-tailed distributions of inertia under uniform seeding. D^2-weighting is a lightweight, principled approach that delivered substantially better and more reliable results on the synthetic benchmarks, with similar total runtime in this experiment and stronger theoretical guarantees for initialisation.

References

[1] D. Arthur and S. Vassilvitskii. *k-means++: The Advantages of Careful Seeding.* SODA, 2007.

[2] S. P. Lloyd. *Least squares quantization in PCM.* IEEE TIT, 1982.

[3] R. Ostrovsky, Y. Rabani, L. Schulman, and C. Swamy. *The effectiveness of Lloyd-type methods for the k-means problem.* FOCS, 2006.

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.